

Guia Rápido

Guia Rápido: Docker & Docker Compose

1. Definição e Propósito

Docker

Docker é uma plataforma de código aberto que automatiza o deployment de aplicações dentro de containers — ambientes leves, portáteis e isolados que empacotam código e todas as suas dependências.

Conceito	Definição
Imagem	Template imutável e read-only com sistema de arquivos, bibliotecas, variáveis de ambiente e configurações. Criada via Dockerfile.
Container	Instância executável de uma imagem. Isolado via namespaces (PID, NET, MNT, UTS, IPC, USER) e limitado via cgroups (CPU, memória, I/O).
Dockerfile	Receita em texto declarando como construir uma imagem (FROM, RUN, COPY, CMD, etc.).
Docker Hub	Registry público oficial de imagens (hub.docker.com).

Isolamento de ambiente: - Cada container tem seu próprio filesystem, processo init, interfaces de rede, montagens e limites de recursos. - Reprodutibilidade: “funciona na minha máquina” deixa de ser um problema. - Imagens em camadas (layer cache) — builds rápidos e reuso.

Docker Compose

Docker Compose é uma ferramenta para definir e rodar aplicações multi-container com um único arquivo YAML (compose.yaml ou docker-compose.yml).

Em projetos reais você nunca roda apenas a aplicação — precisa de banco de dados, cache, fila, proxy reverso, etc. O Compose coordena tudo isso com um comando:

```
docker compose up -d # levanta toda a stack (app + db + redis + ...)
docker compose down # derruba tudo
```

Por que Docker é essencial para Sistemas para Internet?

Cenário real: Você é desenvolvedor em uma equipe de 5 pessoas. Maria usa Windows, João usa Mac, você usa Linux. O banco é PostgreSQL, o cache é Redis, o backend é Node.js, o frontend é React. Sem Docker, cada pessoa passa horas configurando ambiente. Com Docker Compose, um git pull + docker compose up -d e a stack inteira sobe em segundos.

O que Docker resolve na prática:

- **Paridade dev/prod:** o mesmo container que roda no seu computador vai para o servidor. Nada de “funciona na minha máquina”.
- **Onboarding de novos desenvolvedores:** o tutorial de setup vira “instale Docker e rode docker compose up”.
- **Microserviços:** cada serviço em seu container, escala de forma independente.
- **CI/CD:** pipelines sobem containers idênticos para testar, depois promovem para produção.
- **Isolamento de versões:** projeto A usa Node 18, projeto B usa Node 20 — sem conflito algum.

Exemplo de arquitetura web real:

```
[Nginx (proxy)] → [React (container)] → [API Node (container)] → [Postgres (container)]
      ↓
    [Redis (container)]
```

Cada caixa é um container. Cada um escala sozinho. Cada um tem seu Dockerfile. O Compose orquestra tudo.

O repositório github.com/VDCSS/demo-Docker é um exemplo prático desse fluxo: em dois comandos você tem uma aplicação web com banco de dados rodando.

2. Instalação

Docker Engine (Linux — recomendado)

Ubuntu/Debian:

```
sudo apt update && sudo apt install docker.io -y
sudo systemctl enable --now docker
```

Fedora/RHEL:

```
sudo dnf install docker -y
sudo systemctl enable --now docker
```

Script oficial (qualquer distribuição):

```
curl -fsSL https://get.docker.com | sh
```

Pós-instalação (obrigatório):

```
sudo usermod -aG docker $USER
newgrp docker # ou faça logout e login novamente
```

Verificar:

```
docker --version
docker run hello-world
```

Docker Compose Plugin (moderno)

Desde 2023 o Compose é um plugin do Docker CLI. Se instalou via script oficial, já vem incluso.

Instalação manual do plugin:

```
sudo apt install docker-compose-plugin -y
```

Verificar:

```
docker compose version
```

Importante: use docker compose (espaço, plugin) — não confunda com docker-compose (hífen, script Python legado).

3. Comandos Essenciais

Docker

Imagens

```
docker pull nginx:alpine      # baixar imagem
docker images                 # listar imagens locais
docker build -t app:v1 .      # construir imagem
docker rmi app:v1             # remover imagem
```

Containers

```
docker run -d --name app -p 3000:3000 app:v1 # criar e iniciar
docker ps -a                          # listar containers
docker stop app && docker rm app        # parar e remover
docker exec -it app sh                # executar comando
docker logs -f app                    # logs em tempo real
docker inspect app                    # detalhes do container
docker system prune -a                # limpeza geral
```

Flags do docker run: -d (background), -it (interativo), --rm (auto-remover), --name (nome), -p host:cont (porta), -v host:cont (volume), -e VAR=val (ambiente), --network (rede).

Docker Compose

```
docker compose up -d          # subir stack
docker compose up -d --build  # buildar e subir
docker compose down -v       # derrubar + volumes
docker compose ps             # status dos serviços
docker compose logs -f        # logs de todos
docker compose logs -f app    # logs de um serviço
docker compose exec app sh    # shell no serviço
docker compose build          # reconstruir imagens
```

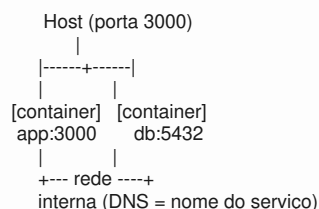
4. Caso Real de Uso em Projeto Web

O repositório abaixo contém uma aplicação Node.js + PostgreSQL pronta para rodar com Docker Compose:

<https://github.com/VDCSS/demo-Docker>

O que a demo faz: uma API em Node.js que consulta a hora atual no banco PostgreSQL e retorna em JSON. Duas rotas — GET / (retorna {"status":"ok","time":"..."}) e GET /health (confirma se o serviço está saudável). O Compose orquestra os dois containers em rede isolada.

Arquitetura



Dois serviços em rede isolada: app (Node.js 20 Alpine, porta 3000) e db (PostgreSQL 16 Alpine, porta 5432). A aplicação se conecta ao banco usando o nome do serviço (db) como hostname.

Para rodar

Pré-requisito: Docker e Docker Compose instalados (consulte a seção 2).

```
git clone https://github.com/VDCSS/demo-Docker.git
cd demo-Docker
docker compose up -d --build
curl http://localhost:3000
# {"status":"ok","time":"2026-06-13T..."}
```

O que cada arquivo faz

Arquivo	Função
app/Dockerfile	imagem Node.js 20 Alpine com layer cache otimizado
app/index.js	servidor Express com duas rotas (/ e /health)
app/package.json	dependências: express e pg
compose.yaml	define os dois serviços, healthcheck no banco e volume nomeado

O README.md do repositório explica cada arquivo em detalhe, incluindo pré-requisitos, endpoints, variáveis de ambiente e comandos úteis.

5. Erros Comuns e Soluções

Erro 1: “Cannot connect to the Docker daemon”

Causa: Usuário sem permissão no grupo docker.

Solução:

```
sudo usermod -aG docker $USER
newgrp docker
```

Erro 2: “port is already allocated”

Causa: Outro container ou processo já usa a porta.

Solução:

```
docker ps | grep 3000
lsof -i :3000
docker run -p 3001:3000 minha-app # porta alternativa
```

Erro 3: ECONNREFUSED ao conectar no banco

Causa: Host errado. Dentro do Compose, use o nome do **serviço** (db), não localhost.

Solução: Verificar DB_HOST. No Compose, localhost refere-se ao próprio container, não ao host.

Erro 4: Imagem não aproveita cache

Causa: Ordem no Dockerfile quebra o cache. Se COPY . . vem antes de RUN npm install, toda alteração no código invalida as dependências.

Solução: Copiar package.json primeiro, instalar, copiar o resto depois:

```
COPY package.json ./
RUN npm install
COPY . .
```

6. Melhores Práticas

- Prefira imagens Alpine (node:20-alpine) — menores e mais seguras
- Use .dockerignore para não copiar node_modules, .git para dentro da imagem
- Copie package.json antes do código para aproveitar o cache de camadas
- Nomeie volumes e redes explicitamente no compose.yaml
- Use healthcheck em serviços dos quais outros dependem

7. Links e Referências Oficiais

Recurso	URL
Documentação oficial Docker	https://docs.docker.com
Documentação Compose	https://docs.docker.com/compose
Dockerfile reference	https://docs.docker.com/reference/dockerfile
Compose file reference	https://docs.docker.com/compose/compose-file
Docker Hub	https://hub.docker.com
Docker Cheat Sheet	https://docs.docker.com/get-started/docker_cheatsheet.pdf
Play with Docker (praticar online)	https://labs.play-with-docker.com
Docker Scout (segurança)	https://docs.docker.com/scout
Awesome Compose (exemplos)	https://github.com/docker/awesome-compose

Resumo (Quick Reference)

```
# --- DOCKER ---
docker build -t app:tag . # construir imagem
docker run -d -p 3000:3000 --name app # rodar container
docker ps # listar containers
docker logs -f app # logs em tempo real
docker exec -it app sh # shell no container
docker system prune -a # limpeza geral

# --- COMPOSE ---
docker compose up -d # subir stack
docker compose down -v # derrubar + volumes
docker compose logs -f # logs agregados
docker compose exec app sh # shell no serviço
docker compose ps # status dos serviços
```

